ALUMNICONNECT

Neo4j Graph Design Specification

Nodes, labels, relationships, properties and traversal patterns

Project: Alumni-Student Mentorship and Mock Interview Platform

Document: Graph data model and implementation specification

Version: 1.0 | Date: 02 November 2026

Design position
Neo4j is a specialised recommendation projection. MySQL remains the authoritative store for identities, skills, availability, requests, allocations, bookings, sessions and feedback. Neo4j discovers relevant relationship paths; MySQL validates business eligibility.

Prepared from the supplied Alumni-Student Mentorship Platform: DBMS Project Blueprint. It expands the blueprint into an implementable Neo4j design, with additional engineering decisions explicitly identified as recommendations.

PES UNIVERSITY | DBMS COURSE PROJECT

Contents

1. Purpose, scope and design decisions 4

1.1 Goals . . . 4

1.2 Scope boundary . . . 4

1.3 Source-derived decisions vs implementation recommendations . . . 4

2. Place of Neo4j in the database architecture 5

2.1 Read path: mentor discovery . . . 5

2.2 Write path: profile changes . . . 5

3. Graph modelling principles 6

3.1 Nodes . . . 6

3.2 Labels . . . 6

3.3 Relationships . . . 6

3.4 Properties . . . 6

3.5 Traversal . . . 6

3.6 Graph projection rather than table replication . . . 6

4. Node labels and property specification 8

4.1 User, Student and Mentor are separate concepts . . . 8

4.2 Property placement examples . . . 9

5. Relationship types, direction and properties 10

5.1 Why relationship properties matter . . . 10

5.2 MENTORED_BY must represent confirmed history . . . 10

5.3 Optional relationships for a later release . . . 10

6. Recommended graph structure and traversal paths 11

6.1 Important path patterns . . . 11

6.2 Direction conventions . . . 11

6.3 Avoid unbounded traversals . . . 11

7. Constraints and indexes 12

7.1 Optional range indexes . . . 12

7.2 Schema checks . . . 12

8. Loading graph data safely 13

8.1 Upsert shared reference nodes . . . 13

8.2 Upsert a mentor and employer link . . . 13

8.3 Upsert a mentor skill snapshot . . . 14

8.4 Upsert a student skill or interest edge . . . 14

9. Small end-to-end sample graph 15

9.1 Create the sample student, mentors and reference nodes . . . 15

9.2 Add the student and mentor skill edges . . . 16

9.3 Add interests and specialisations . . . 16

10. Core Cypher traversal patterns 17

10.1 Find mentors with shared skills . . . 17

10.2 Find mentors aligned to target domains . . . 17

10.3 Find alumni from the same department . . . 17

10.4 Retrieve current employer and job details . . . 17

10.5 Find mentors who guided similar students . . . 18

11. Multi-factor recommendation and explainability 19

11.1 Responsibility split . . . 19
11.2 Candidate score example . . . 19
11.3 Explain the recommendation with evidence . . . 20

12. Recommendation API integration 21

12.1 Suggested API contract . . . 21
12.2 Why recommendation history belongs in MySQL . . . 21

13. Synchronisation: MySQL to Neo4j 22

13.1 Transactional outbox event . . . 22
13.2 Worker processing rules . . . 22
13.3 Example projection event envelope . . . 23
13.4 Failure scenarios and recovery . . . 23

14. Lifecycle rules and graph hygiene 24

14.1 Create . . . 24
14.2 Update . . . 24
14.3 Deactivate . . . 24
14.4 Delete . . . 24
14.5 Rename reference data . . . 24
14.6 Avoid duplicate relationships . . . 24

15. Query performance and tuning 25

15.1 Start from a selective anchor . . . 25
15.2 Keep patterns bounded . . . 25
15.3 Use EXPLAIN and PROFILE . . . 25
15.4 Avoid common performance traps . . . 25

16. Security, privacy and access boundaries 26

16.1 Graph access roles (recommended) . . . 26

17. Testing and acceptance criteria 27

17.1 Demo acceptance checklist . . . 27

18. Implementation sequence 28

18.1 Minimum viable graph release . . . 28
18.2 What should be deferred . . . 28

19. MySQL-to-Neo4j projection mapping 29

19.1 Reconciliation checks . . . 29

20. Final design summary and glossary 30

20.1 Final recommended schema at a glance . . . 30
20.2 Key design decisions to defend in a project review . . . 30
20.3 Glossary . . . 30

Appendix A. Consolidated graph schema checklist 31

Appendix B. Useful Cypher inspection commands . . . 31
Appendix C. Notes on Cypher examples . . . 31

Source basis and implementation references 32

1. Purpose, scope and design decisions

This document specifies the graph data model for AlumniConnect. The graph is designed around the platform capability identified in the project blueprint: discovering suitable alumni through connected students, skills, mentorship domains, companies, departments and prior mentorship relationships.

The goal is not to copy every MySQL table into Neo4j. The graph should represent the connected data needed for relationship-driven discovery and provide explainable paths behind mentor recommendations.

1.1 Goals

- Represent students, alumni mentors and shared reference entities as nodes with stable identifiers.
- Represent skill, domain, department, company and mentorship-history connections as typed relationships.
- Support multi-dimensional mentor discovery rather than single-skill exact matching.
- Return candidate mentor IDs and the relationship evidence that explains each candidate.
- Keep graph updates reliable through a MySQL transactional outbox and idempotent projection worker.
- Keep allocation, booking, availability and other transactional decisions in MySQL.

1.2 Scope boundary

In scope for the Neo4j projection	Not authoritative in Neo4j
Student and mentor graph profiles; shared identity link	Passwords, authentication secrets, access tokens
Student and mentor skill relationships	Authoritative skill catalogue and profile editing
Student domain interests and mentor domain expertise	Mentorship requests and request approval state
Mentor company and department connections	Availability slots, booking locks and calendar conflicts
Confirmed mentorship history for career-path traversal	Booking/session state transitions and feedback records
Graph candidate retrieval and explanation evidence	Final mentor eligibility, allocation and capacity decisions

Non-negotiable rule
A recommendation is not an allocation. A mentor discovered by Neo4j can still be ineligible because of capacity, status, availability, policy or a conflicting allocation. The API must verify those conditions against MySQL before presenting a final actionable result.

1.3 Source-derived decisions vs implementation recommendations

Directly specified by the supplied blueprint: Neo4j represents Student, Mentor, Skill, Domain, Company and Department; it uses the named relationships documented in Section 5 of the blueprint; recommendation dimensions include skill similarity, domain relevance, experience, career relevance, mentor quality and capacity; MySQL remains authoritative; graph synchronization follows a transactional outbox.

Implementation recommendation in this document: add a lightweight User node and profile-link relationships to reflect the blueprint's common identity model; use stable MySQL IDs as graph keys; keep recommendation run history in MySQL; use source-version fields to prevent out-of-order projection updates. These are design choices to make the model safer to implement, not claims that the blueprint already defines these exact Neo4j structures.

2. Place of Neo4j in the database architecture

The blueprint recommends a modular monolith with polyglot persistence. FastAPI coordinates business modules; MySQL stores authoritative relational data; Redis handles cache and temporary low-latency data; Neo4j handles relationship-oriented recommendation queries. Optional vector search is a later extension and is not required for the first graph implementation.

Technology	Role	Authority rule
MySQL 8.x + InnoDB	Primary relational system of record; profiles, skills, requests, allocations, availability, bookings, sessions, feedback and audit data.	Authoritative. Resolve conflicts in favour of MySQL.
Redis	Profile and recommendation cache, short-lived state and rate limiting.	Rebuildable/expiring performance layer; not a booking authority.
Neo4j	Graph projection of student, mentor and reference-entity connections; relationship traversals for recommendations.	Eventually consistent, rebuildable projection.
Redis Stack vector search (optional Phase 2)	Semantic matching of natural-language career goals to profile text.	Search signal only; combine with graph and SQL checks.

2.1 Read path: mentor discovery

The application reads the student profile and eligible criteria from MySQL, asks Neo4j for relationship-based candidates, then validates the candidate IDs against MySQL. The API can combine graph relevance with authoritative ratings, completion outcomes, current capacity and scheduling constraints.

2.2 Write path: profile changes

A profile update is committed in MySQL together with an outbox event in the same SQL transaction. A worker consumes the event and updates Neo4j. Related Redis entries are invalidated. This avoids a fragile dual-write in which an application writes MySQL and Neo4j independently.

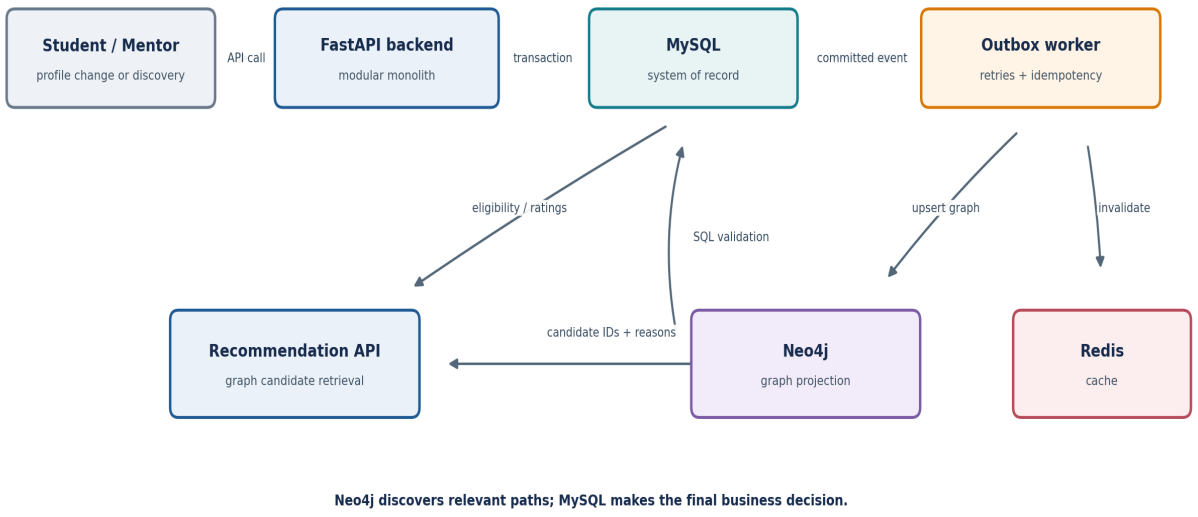


Figure 1. Recommended write and recommendation paths. Neo4j returns relationship candidates; MySQL confirms current business eligibility.

Consistency model
Neo4j may briefly lag behind MySQL. That is acceptable for discovery, but it is not acceptable for booking, allocation or capacity enforcement. Those operations must revalidate and commit in MySQL.

3. Graph modelling principles

3.1 Nodes

A node represents an entity that has an identity in the graph. Examples: one student, one mentor, one skill, one mentorship domain, one company or one department. Each projected node should have a stable ID copied from its corresponding MySQL record.

3.2 Labels

A label classifies a node and helps queries express the type of entity being matched. Use labels such as User, Student, Mentor, Skill, Domain, Company and Department. A node may carry more than one label: a graph identity node may have User plus Student or Mentor labels only if the single-node profile approach is selected. This specification instead separates User from Student and Mentor profile nodes to mirror the project blueprint's common account and role-specific profile concepts.

3.3 Relationships

A relationship is a typed, directed connection between two nodes. Relationship direction should reflect the business meaning, not a visual convenience. For example, the design uses (Student)-[:HAS_SKILL]->(Skill) and (Mentor)-[:HAS_SKILL]->(Skill). A shared skill is found by traversing from the student to a skill and back to a mentor.

3.4 Properties

Properties are key-value attributes on either nodes or relationships. Put stable entity attributes on nodes (for example, mentor job title or skill name). Put facts that describe a particular connection on the relationship (for example, a student's proficiency in a skill or the period of a mentor's employment at a company).

3.5 Traversal

A traversal follows one or more relationships from a starting node to discover connected nodes. Mentor discovery is a natural traversal problem because the answer depends on paths such as Student -> Skill <- Mentor or Student -> Department <- Mentor, not just on independent records.

3.6 Graph projection rather than table replication

- Project only attributes required for discovery, ranking evidence and safe display.
- Do not reproduce all relational columns merely because they exist in MySQL.
- Do not keep a second authoritative copy of request, booking or session state.
- Use IDs, not names, as matching keys. Names can be renamed or duplicated.
- Keep node and relationship types stable; do not create a new relationship type for every skill or domain.

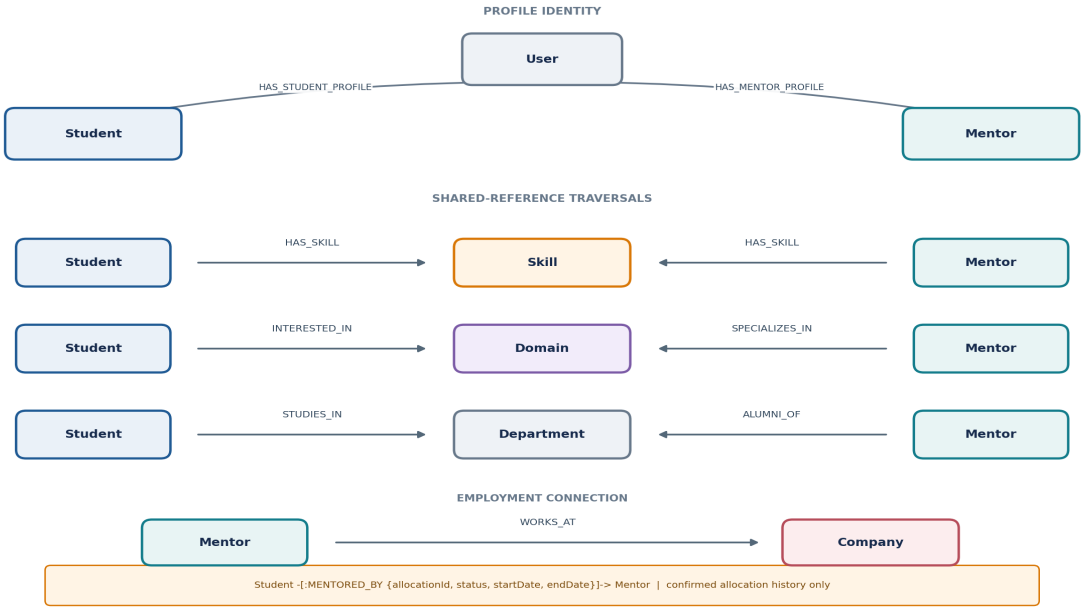


Figure 2. Conceptual graph topology. The central reference nodes (Skill, Domain, Department and Company) create reusable paths between students and mentors.

4. Node labels and property specification

The following schema is the recommended baseline. Property names use lower camel case. IDs are stable identifiers generated by MySQL and carried into Neo4j as strings. Do not use display names as unique identifiers.

Label	Identity property	Recommended properties	Purpose / modelling notes
User	userId	displayName, accountStatus, sourceVersion, updatedAt	Common account identity. Store only safe display identity fields; no password, token or authentication secret.
Student	studentId	userId, programme, graduationYear, careerGoalSummary, isActive, sourceVersion	Student-specific projection. Link to User through HAS_STUDENT_PROFILE.
Mentor	mentorId	userId, jobTitle, yearsExperience, profileSummary, isActive, sourceVersion	Alumni/mentor-specific projection. Link to User through HAS_MENTOR_PROFILE.
Skill	skillId	name, normalizedName, category, isActive, sourceVersion	Shared skill reference node. One node per canonical skill, reused by students and mentors.
Domain	domainId	name, description, isActive, sourceVersion	Mentorship/career domain, such as Machine Learning, MLOps or Cloud Computing.
Company	companyId	name, normalizedName, industry, websiteDomain, isActive, sourceVersion	Employer node. Do not create one node per employment record.
Department	departmentId	name, institution, programmeFamily, isActive, sourceVersion	Academic department / alumni affiliation node.

4.1 User, Student and Mentor are separate concepts

The blueprint recommends a common User identity with separate Student Profile and Mentor Profile concepts. The graph mirrors this by using three node types. A user may have a student profile, a mentor profile, or both if the application permits that role combination. The user link prevents duplicate identity details on the role-specific profiles.

Example 1. Upsert a user and student profile

```
MERGE (u:User {userId: $userId})
SET u.displayName = $displayName,
    u.accountStatus = $accountStatus,
    u.sourceVersion = $sourceVersion,
    u.updatedAt = datetime()

MERGE (s:Student {studentId: $studentId})
SET s.userId = $userId,
    s.programme = $programme,
    s.graduationYear = $graduationYear,
    s.careerGoalSummary = $careerGoalSummary,
    s.isActive = $isActive,
    s.sourceVersion = $sourceVersion
MERGE (u)-[:HAS_STUDENT_PROFILE]->(s)
```

For a mentor profile, use the same pattern with a :Mentor node and :HAS_MENTOR_PROFILE relationship. If one person has both profiles, both profile nodes point to the same :User node. Do not create separate User nodes for the two roles.

4.2 Property placement examples

Fact	Where it belongs	Example
Mentor job title	Mentor node or WORKS_AT relationship depending on whether it describes current role or a specific employer relationship.	Mentor.jobTitle or WORKS_AT.jobTitle
Years of expertise in a particular skill	HAS_SKILL relationship	HAS_SKILL.yearsExperience
Student proficiency in a particular skill	HAS_SKILL relationship	HAS_SKILL.proficiencyLevel
Employment start/end period	WORKS_AT relationship	WORKS_AT.startDate, endDate, isCurrent
Specialisation depth for a domain	SPECIALIZES_IN relationship	SPECIALIZES_IN.experienceYears or level
Mentorship allocation start/end/status	MENTORED_BY relationship	MENTORED_BY.status, startDate, endDate

5. Relationship types, direction and properties

Use the relationship names in the blueprint as the domain vocabulary. The arrows below are directional. Cardinality is a business expectation, not a database-enforced foreign-key rule; application logic and source constraints govern valid records.

Start node	Relationship	End node	Expected multiplicity	Relationship properties
Student	HAS_SKILL	Skill	Student 0..N skills; Skill 0..N students	proficiencyLevel, yearsExperience, acquiredAt, lastVerifiedAt, sourceVersion
Mentor	HAS_SKILL	Skill	Mentor 0..N skills; Skill 0..N mentors	proficiencyLevel, yearsExperience, lastUsedAt, verifiedAt, sourceVersion
Student	INTERESTED_IN	Domain	Student 0..N domains; Domain 0..N students	priority, goalNote, addedAt, sourceVersion
Mentor	SPECIALIZES_IN	Domain	Mentor 0..N domains; Domain 0..N mentors	expertiseLevel, experienceYears, verifiedAt, sourceVersion
Mentor	WORKS_AT	Company	Mentor 0..N employment records; Company 0..N mentors	jobTitle, employmentType, startDate, endDate, isCurrent, sourceVersion
Student	STUDIES_IN	Department	Student typically 0..1 current department; Department 0..N students	programme, startYear, expectedGraduationYear, isCurrent
Mentor	ALUMNI_OF	Department	Mentor 0..N alumni affiliations; Department 0..N mentors	degree, programme, graduationYear, alumniSince
Student	MENTORED_BY	Mentor	Student 0..N confirmed mentor relationships; Mentor 0..N students	allocationId, status, startDate, endDate, sessionsCompleted

5.1 Why relationship properties matter

A skill link has meaning in the context of a particular person. A student can be intermediate in Python while a mentor is advanced in Python. The Skill node should contain the canonical identity of the skill; the HAS_SKILL relationship records the proficiency and experience of that specific person.

Likewise, a mentor may work at the same company at different times or hold different positions. The WORKS_AT relationship stores the employment period and title rather than forcing those facts onto the Company node.

5.2 MENTORED_BY must represent confirmed history

Create MENTORED_BY only after a mentorship allocation has been confirmed by the relational workflow. Do not create it when a student merely views a mentor, receives a recommendation, submits a request, or has a pending request. Keep pending/rejected request state in MySQL.

Example 2. Project a confirmed mentorship allocation

```
MATCH (s:Student {studentId: $studentId})
MATCH (m:Mentor {mentorId: $mentorId})
MERGE (s)-[r:MENTORED_BY {allocationId: $allocationId}]->(m)
SET r.status = $allocationStatus,
    r.startDate = date($startDate),
    r.endDate = CASE WHEN $endDate IS NULL THEN null ELSE date($endDate) END,
    r.sessionsCompleted = $sessionsCompleted
```

5.3 Optional relationships for a later release

- RELATED_TO between Skill nodes can connect canonical synonyms or adjacent skills. Add only with an explicit controlled vocabulary and direction/weight semantics.

- RELATED_TO between Domain nodes can model domain adjacency. It should not replace the authoritative domain taxonomy.
- SIMILAR_TO between profile or goal nodes is not part of the baseline. If introduced, derive it from a defined similarity process and track its algorithm/version.

Do not add a relationship just to make the graph look complex
Every edge should answer a real query or represent a business fact. A graph that contains only meaningful, traversable relationships is easier to explain, test and maintain.

6. Recommended graph structure and traversal paths

The core graph is a bipartite-style matching structure: student and mentor nodes connect to shared Skill, Domain and Department nodes. Mentor nodes additionally connect to Company nodes. Confirmed mentorship history connects Student nodes directly to Mentor nodes.

6.1 Important path patterns

Business question	Graph path	What the path proves
Which mentors share the student's skills?	Student - HAS_SKILL -> Skill <- HAS_SKILL - Mentor	A common skill node joins student and mentor.
Which mentors specialise in the student's target domain?	Student - INTERESTED_IN -> Domain <- SPECIALIZES_IN - Mentor	A mentor specialises in a domain of interest.
Which mentors are alumni of the student's department?	Student - STUDIES_IN -> Department <- ALUMNI_OF - Mentor	A shared academic affiliation exists.
Where does a mentor currently work?	Mentor - WORKS_AT -> Company	Employer, title and employment period are available on the path.
Which mentors guided students with a similar domain interest?	Student <- INTERESTED_IN - Domain <- INTERESTED_IN - Peer - MENTORED_BY -> Mentor	A peer with a shared interest has a confirmed mentoring history with the mentor.
Why was a mentor recommended?	Student -> Skill <- Mentor; Student -> Domain <- Mentor; Student -> Department <- Mentor	The returned path evidence can be converted into user-facing reasons.

6.2 Direction conventions

- Use Student -> Skill and Mentor -> Skill for HAS_SKILL. A shared skill traversal enters Skill from one side and leaves toward the other profile.
- Use Student -> Domain for INTERESTED_IN and Mentor -> Domain for SPECIALIZES_IN.
- Use Student -> Department for STUDIES_IN and Mentor -> Department for ALUMNI_OF.
- Use Mentor -> Company for WORKS_AT.
- Use Student -> Mentor for MENTORED_BY, because the relationship reads as “the student is mentored by this mentor”.

6.3 Avoid unbounded traversals

For the first release, most recommendation paths are one to three relationships long. Prefer explicit patterns such as the ones above. Avoid unrestricted patterns such as [:RELATED_TO*] without a maximum depth. If variable-length traversals are introduced later, specify a bounded range, filter returned paths, and measure the effect with PROFILE.

7. Constraints and indexes

Create constraints before loading graph data. The unique constraints protect stable IDs and give the planner index-backed lookups. The syntax below follows the current Neo4j Cypher constraint form; the uniqueness constraints shown are appropriate for Neo4j editions supporting property uniqueness constraints.

Example 3. Uniqueness constraints for graph identifiers

```
CREATE CONSTRAINT user_id_unique IF NOT EXISTS
FOR (n:User) REQUIRE n.userId IS UNIQUE;

CREATE CONSTRAINT student_id_unique IF NOT EXISTS
FOR (n:Student) REQUIRE n.studentId IS UNIQUE;

CREATE CONSTRAINT mentor_id_unique IF NOT EXISTS
FOR (n:Mentor) REQUIRE n.mentorId IS UNIQUE;

CREATE CONSTRAINT skill_id_unique IF NOT EXISTS
FOR (n:Skill) REQUIRE n.skillId IS UNIQUE;

CREATE CONSTRAINT domain_id_unique IF NOT EXISTS
FOR (n:Domain) REQUIRE n.domainId IS UNIQUE;

CREATE CONSTRAINT company_id_unique IF NOT EXISTS
FOR (n:Company) REQUIRE n.companyId IS UNIQUE;

CREATE CONSTRAINT department_id_unique IF NOT EXISTS
FOR (n:Department) REQUIRE n.departmentId IS UNIQUE;
```

The constraint ensures that two nodes with the same label cannot share the same stable ID. It does not automatically require every node to have that property in all editions/configurations. The import worker must always supply the ID, and the application should validate it before sending data.

7.1 Optional range indexes

Uniqueness constraints already create backing indexes for their constrained properties. Add additional indexes only for properties used frequently in selective filters, for example active mentor filtering. Confirm the plan before retaining an index.

Example 4. Optional lookup indexes

```
CREATE INDEX mentor_active_idx IF NOT EXISTS
FOR (m:Mentor) ON (m.isActive);

CREATE INDEX skill_normalized_name_idx IF NOT EXISTS
FOR (s:Skill) ON (s.normalizedName);

CREATE INDEX domain_name_idx IF NOT EXISTS
FOR (d:Domain) ON (d.name);
```

7.2 Schema checks

Example 5. Verify constraints, indexes and graph vocabulary

```
SHOW CONSTRAINTS;
SHOW INDEXES;

// Inspect the labels and relationship types in the graph
CALL db.labels();
CALL db.relationshipTypes();
```

Enterprise edition note
Property existence constraints, property type constraints and node-key constraints have edition/version considerations. The baseline design relies on property uniqueness constraints plus validation in the ingestion worker. Do not assume an Enterprise-only constraint is available in a Community deployment.

8. Loading graph data safely

Use MERGE for idempotent projection writes. Create constraints first. MERGE identifies the node or relationship by its declared pattern; SET refreshes non-key properties. Avoid putting all mutable profile properties into a MERGE pattern, because a changed property would make the pattern fail to match the original node.

8.1 Upsert shared reference nodes

Example 6. Upsert canonical Skill and Domain nodes

```
UNWIND $skills AS item
MERGE (sk:Skill {skillId: item.skillId})
SET sk.name = item.name,
    sk.normalizedName = item.normalizedName,
    sk.category = item.category,
    sk.isActive = item.isActive,
    sk.sourceVersion = item.sourceVersion;

UNWIND $domains AS item
MERGE (d:Domain {domainId: item.domainId})
SET d.name = item.name,
    d.description = item.description,
    d.isActive = item.isActive,
    d.sourceVersion = item.sourceVersion;
```

8.2 Upsert a mentor and employer link

Example 7. Upsert a mentor profile and employment edge

```
MERGE (u:User {userId: $userId})
SET u.displayName = $displayName,
    u.accountStatus = $accountStatus,
    u.sourceVersion = $userVersion

MERGE (m:Mentor {mentorId: $mentorId})
SET m.userId = $userId,
    m.jobTitle = $jobTitle,
    m.yearsExperience = $yearsExperience,
    m.profileSummary = $profileSummary,
    m.isActive = $isActive,
    m.sourceVersion = $mentorVersion
MERGE (u)-[:HAS_MENTOR_PROFILE]->(m)

MERGE (c:Company {companyId: $companyId})
SET c.name = $companyName,
    c.industry = $industry,
    c.isActive = true
MERGE (m)-[w:WORKS_AT {employmentKey: $employmentKey}]->(c)
SET w.jobTitle = $jobTitle,
    w.startDate = date($employmentStart),
    w.isCurrent = $isCurrent,
    w.endDate = CASE WHEN $employmentEnd IS NULL THEN null ELSE date($employmentEnd) END;
```

The `employmentKey` is a stable source-side identifier for one employment record. If the relational schema does not have a dedicated employment ID, construct and persist a stable projection key in the outbox payload rather than relying on company name or job title.

8.3 Upsert a mentor skill snapshot

Example 8. Replace a mentor skill snapshot only when the event is newer

```
MATCH (m:Mentor {mentorId: $mentorId})
WHERE coalesce(m.sourceVersion, 0) < $sourceVersion

OPTIONAL MATCH (m)-[old:HAS_SKILL]->(:Skill)
WITH m, collect(old) AS oldRelationships
FOREACH (r IN oldRelationships | DELETE r)
SET m.sourceVersion = $sourceVersion

WITH m
UNWIND $skills AS item
MERGE (sk:Skill {skillId: item.skillId})
ON CREATE SET sk.name = item.name,
              sk.normalizedName = item.normalizedName
MERGE (m)-[r:HAS_SKILL]->(sk)
SET r.proficiencyLevel = item.proficiencyLevel,
    r.yearsExperience = item.yearsExperience,
    r.verifiedAt = CASE WHEN item.verifiedAt IS NULL
                        THEN null ELSE datetime(item.verifiedAt) END,
    r.sourceVersion = $sourceVersion;
```

This sample assumes the worker sends a complete snapshot of that mentor's skills. It first removes the old HAS_SKILL relationships and then writes the current set. If the source event is a delta rather than a full snapshot, use explicit add/remove operations instead. The `sourceVersion` guard prevents an older event from overwriting a newer graph projection. A zero-skill snapshot still applies the deletion and sourceVersion update because those clauses execute before UNWIND. The worker should treat the resulting zero-row write as a successful projection update and acknowledge the event only after the Neo4j transaction commits.

8.4 Upsert a student skill or interest edge

Example 9. Create student skill and domain-interest connections

```
MATCH (s:Student {studentId: $studentId})
MATCH (sk:Skill {skillId: $skillId})
MERGE (s)-[r:HAS_SKILL]->(sk)
SET r.proficiencyLevel = $proficiencyLevel,
    r.yearsExperience = $yearsExperience,
    r.acquiredAt = date($acquiredAt),
    r.sourceVersion = $sourceVersion;

MATCH (s:Student {studentId: $studentId})
MATCH (d:Domain {domainId: $domainId})
MERGE (s)-[r:INTERESTED_IN]->(d)
SET r.priority = $priority,
    r.goalNote = $goalNote,
    r.sourceVersion = $sourceVersion;
```

9. Small end-to-end sample graph

The blueprint illustrates a Student A interested in Machine Learning, MLOps and Cloud Computing, with Python, Machine Learning and SQL skills. It then compares three example mentors with different skill/domain profiles. The following sample uses those same conceptual profiles and neutral IDs so the graph can be loaded and queried during a demo.

9.1 Create the sample student, mentors and reference nodes

Example 10. Seed the sample graph

```
// Student A
MERGE (uS:User {userId: 'U-001'})
SET uS.displayName = 'Student A'
MERGE (s:Student {studentId: 'STU-001'})
SET s.userId = 'U-001', s.isActive = true,
    s.careerGoalSummary = 'Machine learning and production ML systems'
MERGE (uS)-[:HAS_STUDENT_PROFILE]->(s);

// Three example mentors
UNWIND [
  {userId:'U-101', mentorId:'MEN-001', name:'Mentor A',
   title:'ML Engineer', years:5},
  {userId:'U-102', mentorId:'MEN-002', name:'Mentor B',
   title:'Cloud Engineer', years:4},
  {userId:'U-103', mentorId:'MEN-003', name:'Mentor C',
   title:'Data Scientist', years:3}
] AS row
MERGE (u:User {userId: row.userId}) SET u.displayName = row.name
MERGE (m:Mentor {mentorId: row.mentorId})
SET m.userId = row.userId, m.jobTitle = row.title,
    m.yearsExperience = row.years, m.isActive = true
MERGE (u)-[:HAS_MENTOR_PROFILE]->(m);

// Canonical skills and domains
UNWIND ['Python','Machine Learning','SQL','MLOps','Docker',
        'AWS','Kubernetes','Data Science'] AS skillName
MERGE (sk:Skill {skillId: toUpper(replace(skillName,' ','-'))})
SET sk.name = skillName;

UNWIND ['Machine Learning','MLOps','Cloud Computing','Data Analytics'] AS domainName
MERGE (d:Domain {domainId: toUpper(replace(domainName,' ','-'))})
SET d.name = domainName;
```

In an actual project, skillId and domainId should come from MySQL. The sample uses deterministic IDs for a compact classroom demonstration. The source system should normalise names and merge aliases before graph ingestion.

9.2 Add the student and mentor skill edges

Example 11. Add the skills for the four profiles

```
MATCH (s:Student {studentId:'STU-001'})
UNWIND ['Python','Machine Learning','SQL'] AS n
MATCH (sk:Skill {name:n})
MERGE (s)-[r:HAS_SKILL]->(sk)
SET r.proficiencyLevel = 'INTERMEDIATE';

MATCH (m:Mentor {mentorId:'MEN-001'})
UNWIND [
 {name:'Python', level:'ADVANCED'},
 {name:'Machine Learning', level:'ADVANCED'},
 {name:'MLOps', level:'ADVANCED'},
 {name:'Docker', level:'INTERMEDIATE'}
] AS item
MATCH (sk:Skill {name:item.name})
MERGE (m)-[r:HAS_SKILL]->(sk)
SET r.proficiencyLevel = item.level;

MATCH (m:Mentor {mentorId:'MEN-002'})
UNWIND ['AWS','Docker','Kubernetes'] AS n
MATCH (sk:Skill {name:n})
MERGE (m)-[:HAS_SKILL]->(sk);

MATCH (m:Mentor {mentorId:'MEN-003'})
UNWIND ['Python','SQL','Data Science'] AS n
MATCH (sk:Skill {name:n})
MERGE (m)-[:HAS_SKILL]->(sk);
```

9.3 Add interests and specialisations

Example 12. Add student interests and mentor specialisations

```
MATCH (s:Student {studentId:'STU-001'})
UNWIND ['Machine Learning','MLOps','Cloud Computing'] AS n
MATCH (d:Domain {name:n})
MERGE (s)-[:INTERESTED_IN]->(d);

MATCH (m:Mentor {mentorId:'MEN-001'})
MATCH (d:Domain {name:'Machine Learning'})
MERGE (m)-[:SPECIALIZES_IN {experienceYears:5}]->(d);

MATCH (m:Mentor {mentorId:'MEN-002'})
MATCH (d:Domain {name:'Cloud Computing'})
MERGE (m)-[:SPECIALIZES_IN {experienceYears:4}]->(d);

MATCH (m:Mentor {mentorId:'MEN-003'})
MATCH (d:Domain {name:'Data Analytics'})
MERGE (m)-[:SPECIALIZES_IN {experienceYears:3}]->(d);
```

Expected qualitative result
Mentor A has the strongest direct overlap for Student A: shared Python and Machine Learning skills plus a Machine Learning specialisation. Mentor B is relevant through Cloud Computing interest and infrastructure skills, although the student does not yet share AWS or Kubernetes. Mentor C shares Python and SQL but specialises in Data Analytics, which is less directly aligned with the listed interests.

10. Core Cypher traversal patterns

The examples use parameters such as `$studentId`. Bind parameters from the backend driver; do not concatenate raw user input into Cypher text. Each query returns stable mentor IDs so the API can validate the candidates in MySQL.

10.1 Find mentors with shared skills

Pattern A. Skill overlap

```
MATCH (s:Student {studentId: $studentId})-[:HAS_SKILL]->(sk:Skill)
      <-[:HAS_SKILL]-(m:Mentor)
WHERE m.isActive = true
WITH m, collect(DISTINCT sk.name) AS sharedSkills
RETURN m.mentorId AS mentorId,
       m.jobTitle AS jobTitle,
       m.yearsExperience AS yearsExperience,
       sharedSkills,
       size(sharedSkills) AS sharedSkillCount
ORDER BY sharedSkillCount DESC, yearsExperience DESC
LIMIT $limit;
```

Use this when the student wants a mentor who knows tools or concepts they already use. The `collect(DISTINCT ...)` prevents duplicate evidence from inflating the count if the graph contains multiple paths to the same skill.

10.2 Find mentors aligned to target domains

Pattern B. Domain relevance

```
MATCH (s:Student {studentId: $studentId})-[:INTERESTED_IN]->(d:Domain)
      <-[:SPECIALIZES_IN]-(m:Mentor)
WHERE m.isActive = true
WITH m, collect(DISTINCT d.name) AS matchingDomains
RETURN m.mentorId AS mentorId,
       matchingDomains,
       size(matchingDomains) AS matchingDomainCount
ORDER BY matchingDomainCount DESC, m.yearsExperience DESC
LIMIT $limit;
```

10.3 Find alumni from the same department

Pattern C. Shared academic affiliation

```
MATCH (s:Student {studentId: $studentId})-[:STUDIES_IN]->(d:Department)
      <-[:ALUMNI_OF]-(m:Mentor)
WHERE m.isActive = true
RETURN DISTINCT m.mentorId AS mentorId,
       d.name AS sharedDepartment,
       m.jobTitle AS jobTitle,
       m.yearsExperience AS yearsExperience
ORDER BY yearsExperience DESC
LIMIT $limit;
```

10.4 Retrieve current employer and job details

Pattern D. Mentor industry context

```
MATCH (m:Mentor {mentorId: $mentorId})-[w:WORKS_AT]->(c:Company)
WHERE w.isCurrent = true
RETURN c.companyId AS companyId,
       c.name AS company,
       c.industry AS industry,
       w.jobTitle AS jobTitle,
       w.startDate AS startDate;
```

10.5 Find mentors who guided similar students

Pattern E. Indirect career/mentorship connections

```
MATCH (s:Student {studentId: $studentId})-[:INTERESTED_IN]->(d:Domain)
      <-[:INTERESTED_IN]-(peer:Student)
MATCH (peer)-[history:MENTORED_BY]->(m:Mentor)
WHERE m.isActive = true
  AND history.status IN ['ACTIVE', 'COMPLETED']
  AND peer.studentId <> s.studentId
RETURN m.mentorId AS mentorId,
       collect(DISTINCT peer.studentId) AS similarStudents,
       collect(DISTINCT d.name) AS sharedDomains,
       count(DISTINCT history.allocationId) AS relevantMentorships
ORDER BY relevantMentorships DESC
LIMIT $limit;
```

This pattern should use only confirmed allocations represented by MENTORED_BY. The allowed status values should match the status vocabulary defined by the project. It is a signal of prior mentoring experience with students who share an interest, not proof that the mentor is currently available.

11. Multi-factor recommendation and explainability

The blueprint explicitly advises that recommendation should not be based on one matching skill. It lists six dimensions: skill similarity, domain relevance, experience, career relevance, mentor quality and capacity. It also says scoring weights should be configurable rather than permanently hard-coded.

11.1 Responsibility split

Signal	Best source	Why
Shared skills	Neo4j traversal	Graph can count and return the common skill nodes.
Domain relevance	Neo4j traversal	Graph joins student interests to mentor specialisations.
Industry / experience relevance	Neo4j for relationships; MySQL for authoritative profile facts	Graph can follow WORKS_AT and profile connections; MySQL holds the canonical employment/profile record.
Career relevance	Neo4j relationship paths + optional vector search later	Paths and domain connections give explainable evidence; semantic similarity can be a Phase 2 signal.
Mentor quality	MySQL	Ratings, feedback and session outcomes remain authoritative relational data.
Capacity and current eligibility	MySQL	Must reflect current allocations, availability, restrictions and policy at decision time.

11.2 Candidate score example

The following is a graph relevance score, not a probability and not a final eligibility result. Use configurable parameters from the backend so the weights can be tuned and evaluated.

Example 13. Configurable graph candidate score

```
MATCH (s:Student {studentId: $studentId})
MATCH (m:Mentor)
WHERE m.isActive = true

OPTIONAL MATCH (s)-[:HAS_SKILL]->(skill:Skill)<-[:HAS_SKILL]-(m)
WITH s, m, collect(DISTINCT skill.name) AS sharedSkills

OPTIONAL MATCH (s)-[:INTERESTED_IN]->(domain:Domain)
             <-[:SPECIALIZES_IN]-(m)
WITH s, m, sharedSkills,
     collect(DISTINCT domain.name) AS matchingDomains

OPTIONAL MATCH (s)-[:STUDIES_IN]->(dept:Department)
              <-[:ALUMNI_OF]-(m)
WITH m, sharedSkills, matchingDomains,
     collect(DISTINCT dept.name) AS sharedDepartments

WITH m, sharedSkills, matchingDomains, sharedDepartments,
     size(sharedSkills) * $skillWeight
   + size(matchingDomains) * $domainWeight
   + size(sharedDepartments) * $alumniWeight
   + CASE WHEN m.yearsExperience >= $experienceThreshold
          THEN $experienceBonus ELSE 0 END AS graphScore
WHERE size(sharedSkills) + size(matchingDomains)
      + size(sharedDepartments) > 0
RETURN m.mentorId AS mentorId,
       graphScore,
       sharedSkills,
       matchingDomains,
       sharedDepartments,
       m.yearsExperience AS yearsExperience
ORDER BY graphScore DESC, yearsExperience DESC
LIMIT $candidateLimit;
```

Example parameter set for a classroom run: `skillWeight = 3.0`, `domainWeight = 5.0`, `alumniWeight = 1.0`, `experienceThreshold = 4`, `experienceBonus = 2.0`. These are illustrative starting values only. Evaluate rankings using labelled test cases and adjust them; do not present the score as a calibrated percentage.

11.3 Explain the recommendation with evidence

Return the evidence behind a candidate, not just the score. For example: “Shares Python and Machine Learning; specialises in Machine Learning; has a shared Computer Science department affiliation; has five years of experience.” The API can turn the returned lists into concise reasons.

Example 14. Build an explanation payload for one mentor

```
// A focused explanation query for a selected student/mentor pair
MATCH (s:Student {studentId: $studentId})
MATCH (m:Mentor {mentorId: $mentorId})
OPTIONAL MATCH (s)-[:HAS_SKILL]->(sk:Skill)<-[:HAS_SKILL]-(m)
WITH s, m, collect(DISTINCT sk.name) AS sharedSkills
OPTIONAL MATCH (s)-[:INTERESTED_IN]->(d:Domain)
              <-[:SPECIALIZES_IN]-(m)
WITH s, m, sharedSkills, collect(DISTINCT d.name) AS matchingDomains
OPTIONAL MATCH (s)-[:STUDIES_IN]->(dep:Department)
              <-[:ALUMNI_OF]-(m)
RETURN m.mentorId AS mentorId,
       sharedSkills,
       matchingDomains,
       collect(DISTINCT dep.name) AS sharedDepartments,
       m.yearsExperience AS yearsExperience;
```

12. Recommendation API integration

Treat Neo4j as candidate generation and evidence retrieval. The backend owns orchestration and final decision-making. A recommended request flow is:

Step 1 - Read student context. Load the student profile, active skills, target domains and policy filters from MySQL.

Step 2 - Retrieve graph candidates. Run the multi-factor Cypher traversal and request a candidate set larger than the final page size (for example, top 30 candidates for a final top 10).

Step 3 - Validate candidates in MySQL. Confirm that the mentor is active, eligible, not blocked, has remaining capacity, meets role/domain rules and is not disqualified by current allocations or policy.

Step 4 - Enrich ranking. Retrieve authoritative rating aggregates, feedback quality signals, completed-session outcomes and other non-graph scoring fields from MySQL.

Step 5 - Produce explanations. Convert graph evidence into reason strings and combine it with SQL-derived signals.

Step 6 - Persist recommendation history. Store the recommendation run and candidate snapshot in MySQL if the project needs to answer what was recommended at a particular point in time.

Step 7 - Cache carefully. Cache results in Redis with a short TTL and invalidate them when student skills, mentor skills, domains or eligibility-relevant fields change.

12.1 Suggested API contract

Illustrative response shape

```
GET /api/v1/students/{studentId}/mentor-recommendations?limit=10

Response (illustrative):
{
  "studentId": "STU-001",
  "generatedAt": "2026-11-02T12:00:00Z",
  "recommendations": [
    {
      "mentorId": "MEN-001",
      "rank": 1,
      "graphScore": 17.0,
      "evidence": {
        "sharedSkills": ["Python", "Machine Learning"],
        "matchingDomains": ["Machine Learning"],
        "sharedDepartments": ["Computer Science"]
      },
      "eligibility": "VERIFIED_BY_MYSQL"
    }
  ]
}
```

The response intentionally does not expose graph internals or assume a 17-point score is a 17% likelihood. The application should return only fields that are safe and useful to the requesting student.

12.2 Why recommendation history belongs in MySQL

The blueprint proposes Recommendation Run and Candidate Recommendation concepts to retain recommendation history and explainability. Keep the audit snapshot in MySQL because it is part of the application record and may need to be reported consistently with the relational workflow. Neo4j remains the graph engine used to compute current candidate paths.

13. Synchronisation: MySQL to Neo4j

The blueprint specifies a six-step flow: commit the profile update in MySQL, record an outbox event in the same transaction, let a background worker read the event, update Neo4j, invalidate relevant Redis cache entries, then allow recommendations to use the updated graph. This section turns that flow into operational rules.

13.1 Transactional outbox event

A profile update transaction should write both the business row changes and a small outbox event. If the transaction rolls back, neither the profile update nor the event is visible. A worker polls or subscribes to committed outbox events and applies them to Neo4j.

Outbox field	Example	Purpose
eventId	EVT-000045	Unique event identity for idempotent processing and logs.
aggregateType	MENTOR	Identifies the entity type.
aggregateId	MEN-001	Identifies the source entity.
eventType	MentorSkillsReplaced	Names the projection action.
aggregateVersion	42	Monotonic source version for ordering and stale-event protection.
occurredAt	2026-11-02T12:00:00Z	Source event time; useful for diagnostics.
payload	complete skill snapshot	Contains the minimum information the projection worker needs.

13.2 Worker processing rules

- Read the event and payload from the committed MySQL outbox.
- Validate the event schema and all stable IDs before issuing Cypher.
- Use a Neo4j transaction for each logical projection update or bounded batch.
- Make the update idempotent: reprocessing the same event must not duplicate nodes or relationships.
- Apply sourceVersion checks so an older event cannot overwrite newer state.
- After Neo4j commits, mark the outbox event delivered. If it fails, retry with backoff and retain an error record.
- Invalidate related Redis keys only after the MySQL change is committed; retries should be safe.

13.3 Example projection event envelope

Example 15. Outbox payload for a full skill snapshot

```
{
  "eventId": "EVT-000045",
  "eventType": "MentorSkillsReplaced",
  "aggregateType": "MENTOR",
  "aggregateId": "MEN-001",
  "aggregateVersion": 42,
  "occurredAt": "2026-11-02T12:00:00Z",
  "payload": {
    "mentorId": "MEN-001",
    "skills": [
      {
        "skillId": "SK-PYTHON",
        "name": "Python",
        "proficiencyLevel": "ADVANCED",
        "yearsExperience": 4
      },
      {
        "skillId": "SK-DOCKER",
        "name": "Docker",
        "proficiencyLevel": "INTERMEDIATE",
        "yearsExperience": 2
      }
    ]
  }
}
```

13.4 Failure scenarios and recovery

Failure	Expected response
Worker crashes before Neo4j commit	Do not mark the event delivered. Retry the same event.
Worker commits Neo4j but crashes before outbox acknowledgement	The event may replay. MERGE + sourceVersion makes replay safe.
Events arrive out of order	Reject or ignore the event when aggregateVersion is not newer than the graph node version.
Neo4j is unavailable	Keep events pending in MySQL; retry. MySQL business transactions continue if graph projection is not required for the write.
Graph node/relationship was corrupted or deleted	Run reconciliation or rebuild the projection from MySQL snapshots.
A user is removed or deactivated	Apply the explicit deactivation/deletion policy and remove or disable associated graph paths.

14. Lifecycle rules and graph hygiene

14.1 Create

Create the node from a source-of-truth record, then create the relevant relationships once both endpoints are known. Use MERGE on stable identifiers, not names.

14.2 Update

Update mutable node attributes with SET. Update relationship properties when the source relationship changes. For a full collection update (for example, all current mentor skills), remove stale edges and recreate the current set in one idempotent projection operation.

14.3 Deactivate

A profile deactivated in MySQL should no longer be recommended. Mirror the state with `isActive = false` and make every candidate query filter active mentors. This is a discovery filter, not a substitute for MySQL eligibility validation.

14.4 Delete

Prefer an explicit soft-delete/deactivation projection when historical mentorship paths or auditability matter. Physically delete a graph node only when the source deletion policy requires it and no retained history needs the node. `DETACH DELETE` removes all incident relationships, so use it only after confirming that all downstream graph paths can be discarded.

14.5 Rename reference data

A Skill or Domain rename should update the existing node by its stable ID. Do not create a new node because a label spelling changed. If two reference records are merged in MySQL, update the projection so old edges point to the canonical node, then remove the obsolete node after verifying it is unused.

14.6 Avoid duplicate relationships

For relationship types whose endpoints can have at most one logical edge, use `MERGE (a)-[r:TYPE]->(b)` and then set properties. Where multiple real-world records may exist between the same endpoint pair (such as repeated employment periods or multiple allocations), include a stable source record key in the relationship pattern, for example `employmentKey` or `allocationId`.

15. Query performance and tuning

15.1 Start from a selective anchor

Anchor queries on an indexed stable ID such as `Student.studentId` or `Mentor.mentorId`. Avoid starting with every Skill or every Mentor node and then filtering late. Unique constraints on IDs should support selective lookups.

15.2 Keep patterns bounded

- Use explicit path patterns for the most common match dimensions.
- Filter inactive profiles and inactive reference nodes as early as practical.
- Collect distinct node IDs/names so multiple paths do not overcount the same evidence.
- Return only the fields needed by the API, not full nodes or entire paths by default.
- Set a candidate limit; then apply authoritative SQL eligibility and ranking.

15.3 Use EXPLAIN and PROFILE

Use EXPLAIN to inspect the planned operators without executing the query. Use PROFILE in a controlled test environment to execute and inspect actual rows, database hits and operator work. Compare the query plan before and after adding an index or changing the traversal shape. Record the test dataset size and Neo4j version with every benchmark.

Example 16. Inspect a skill-overlap query plan

```
EXPLAIN
MATCH (s:Student {studentId: $studentId})-[:HAS_SKILL]->(sk:Skill)
      <-[:HAS_SKILL]-(m:Mentor)
WHERE m.isActive = true
RETURN DISTINCT m.mentorId;

PROFILE
MATCH (s:Student {studentId: $studentId})-[:HAS_SKILL]->(sk:Skill)
      <-[:HAS_SKILL]-(m:Mentor)
WHERE m.isActive = true
RETURN DISTINCT m.mentorId;
```

15.4 Avoid common performance traps

Anti-pattern	Better approach
Using names as identifiers	Use unique IDs and keep names as display properties.
Unbounded variable-length traversal	Set a maximum depth and define what a path means.
Returning every matching path	Aggregate evidence and return candidate IDs plus reasons.
Storing large biographies on nodes and returning them repeatedly	Keep only a short summary if needed; fetch full profile text from MySQL when selected.
Adding indexes without measuring	Use PROFILE and retain only indexes supporting real filters.
Overlapping duplicate edges	Use MERGE and stable relationship source keys where several records are valid.
Scoring all mentors before narrowing by relevance	Use graph paths to discover a bounded candidate set first.

16. Security, privacy and access boundaries

The graph is a recommendation projection, not a second user-management database. Apply data minimisation: replicate only the profile facts needed for candidate matching and safe display.

- Never project passwords, password hashes, access tokens, refresh tokens, OTP secrets or authentication challenge values into Neo4j.
- Avoid copying private contact information unless a documented use case and access policy require it. Prefer a public display name and profile identifier.
- Keep free-text feedback and private review comments in MySQL; recommendation queries normally need aggregate quality signals, not the original text.
- Use a dedicated Neo4j application account with the least privileges required for read/write projection operations.
- Use TLS for database connections where applicable, store credentials in environment secrets, and never commit passwords to the repository.
- Parameterise Cypher values. Do not interpolate user-controlled strings into query text or dynamic labels/relationship types.
- Log event IDs, source IDs and processing status. Avoid logging unnecessary personal data or entire profile payloads.

16.1 Graph access roles (recommended)

Application component	Required graph access
Projection worker	Create/update the allowed labels and relationships; no administrative schema privileges at runtime.
Recommendation API	Read-only access to the graph projection.
Developer / migration role	Schema setup, constraint/index management and controlled troubleshooting.
End user / browser client	No direct database access. All requests go through the backend API.

Privacy principle
Do not return a graph path merely because it exists. A recommendation explanation should contain only evidence the requesting student is authorised to see.

17. Testing and acceptance criteria

The graph implementation should be tested as a projection and as a recommendation feature. Tests should cover node identity, relationship correctness, idempotency, traversal results, SQL eligibility filtering and recovery after failures.

Test ID	Scenario	Expected result
G-01	Create a User with Student profile	Exactly one User node and one Student node; one HAS_STUDENT_PROFILE edge.
G-02	Same account also has a Mentor profile	Both profile nodes point to the same User node; no duplicate User node.
G-03	Student and mentor share Python	Skill traversal returns that mentor with Python in sharedSkills.
G-04	Student is interested in MLOps; mentor specialises in MLOps	Domain traversal returns the mentor and MLOps evidence.
G-05	Mentor belongs to the student's department	Department traversal returns the mentor with the shared department.
G-06	Same outbox event is processed twice	No duplicate nodes or logical relationships; state remains correct.
G-07	Older sourceVersion arrives after newer event	Older event is ignored; newer projection state remains.
G-08	Mentor deactivated in MySQL and projected	Neo4j queries do not return the inactive mentor.
G-09	Graph returns a full-capacity mentor	MySQL eligibility check excludes the mentor from final actionable results.
G-10	Neo4j is unavailable during a profile update	MySQL transaction commits with pending outbox event; projection is retried later.
G-11	Full projection rebuild	Graph nodes and edges can be reconstructed from MySQL without losing authoritative business data.
G-12	Recommendation response includes explanation	Each displayed reason is supported by returned graph evidence or SQL-derived signals.

17.1 Demo acceptance checklist

- Show Neo4j Browser with the Student, Mentor, Skill, Domain, Company and Department labels.
- Run one query for shared skill matching, one for domain matching and one for same-department alumni.
- Show the multi-factor candidate ranking and evidence returned for the top candidates.
- Demonstrate that a mentor at capacity is removed by MySQL even when Neo4j recommends them.
- Update a mentor skill in MySQL, inspect the outbox event, then show the new HAS_SKILL edge in Neo4j.
- Replay the same event to demonstrate idempotency; replay an older event to demonstrate version protection.
- Show EXPLAIN/PROFILE output and explain the purpose of the unique ID constraints.

18. Implementation sequence

Phase	Work	Deliverable / exit condition
1. Model	Confirm source IDs, labels, node properties and relationship vocabulary with the MySQL schema.	Approved graph schema and mapping sheet.
2. Local graph	Create database, uniqueness constraints and required indexes. Load a small test dataset.	Nodes and edges visible in Neo4j Browser; no duplicate IDs.
3. Projection worker	Implement outbox event consumption, idempotent MERGE operations, version checks and retries.	MySQL updates eventually appear in Neo4j.
4. Traversal queries	Implement skill, domain, department, company and mentorship-history queries.	Cypher query tests pass with deterministic sample data.
5. Recommendation API	Combine Neo4j evidence with MySQL capacity, ratings, status and policy checks.	API returns eligible candidates and explanations.
6. Redis integration	Cache recommendation responses; invalidate after relevant source changes.	Cache hit/miss/expiry and invalidation demo works.
7. Evaluation	Test ranking quality, query plans, event replay, rebuild and failure handling.	Acceptance checklist completed; limitations documented.

18.1 Minimum viable graph release

For the first working submission, implement User, Student, Mentor, Skill and Domain nodes, with HAS_SKILL, INTERESTED_IN and SPECIALIZES_IN. Add Department, Company, STUDIES_IN, ALUMNI_OF and WORKS_AT once the initial recommendation queries pass. Add MENTORED_BY only when the allocation workflow can reliably publish confirmed allocation history.

18.2 What should be deferred

- Semantic/vector similarity is a Phase 2 extension in the blueprint; do not block the graph MVP on it.
- Do not model bookings, availability slots or every session as graph nodes for the first release. They are transactional MySQL concerns.
- Do not introduce complex path algorithms, graph data science or inferred similarity edges until simple traversal baselines are measured.

19. MySQL-to-Neo4j projection mapping

The source blueprint identifies the existing mini-project entities and then proposes a broader conceptual model. The mapping below is a practical projection guide; exact table and column names must be aligned with the final relational schema.

Relational source concept	Neo4j representation	Projection rule
User / account	(:User {userId})	One node per account; only safe display/status fields.
Student profile	(:Student {studentId}) and HAS_STUDENT_PROFILE	Project student profile attributes needed for discovery.
Alumni / mentor profile	(:Mentor {mentorId}) and HAS_MENTOR_PROFILE	Project professional summary, active state and experience signals.
Student Skill junction	(Student)-[:HAS_SKILL]->(Skill)	Junction attributes become edge properties.
Alumni Skill junction	(Mentor)-[:HAS_SKILL]->(Skill)	Junction attributes become edge properties.
Mentorship Domain / interest table	Domain nodes plus INTERESTED_IN / SPECIALIZES_IN	Use canonical domain IDs; priority/expertise belongs on the edge.
Company / employment record	Company node plus WORKS_AT	One company node; one relationship per employment record.
Department / affiliation	Department node plus STUDIES_IN / ALUMNI_OF	Retain graduation/programme facts on the relationship where applicable.
Mentor Allocation	MENTORED_BY relationship	Only confirmed allocation records; relationship keyed by allocationId.
Request, availability, booking, session, feedback	Remain in MySQL for baseline release	Do not use graph projection as transactional state or capacity authority.

19.1 Reconciliation checks

- For every active Student in MySQL intended for projection, confirm one Student graph node by studentId.
- For every active Mentor in MySQL intended for projection, confirm one Mentor graph node by mentorId.
- Compare counts of active student-skill and mentor-skill links between MySQL and Neo4j.
- Check for dangling profile relationships and orphaned reference nodes.
- Verify every MENTORED_BY allocationId exists in the confirmed allocation source set.
- Store the last successful full rebuild time and last processed outbox event/version.

20. Final design summary and glossary

20.1 Final recommended schema at a glance

Nodes: User, Student, Mentor, Skill, Domain, Company, Department.

Relationships: HAS_STUDENT_PROFILE, HAS_MENTOR_PROFILE, HAS_SKILL, INTERESTED_IN, SPECIALIZES_IN, WORKS_AT, STUDIES_IN, ALUMNI_OF, MENTORED_BY.

Core traversals: Student -> Skill <- Mentor; Student -> Domain <- Mentor; Student -> Department <- Mentor; Student <- Domain <- Peer -> Mentor through confirmed mentorship history; Mentor -> Company.

Operational model: MySQL source of truth; transactional outbox; idempotent worker; eventually consistent Neo4j projection; Redis cache invalidation; MySQL final eligibility checks.

20.2 Key design decisions to defend in a project review

- Why use Neo4j? Mentor discovery depends on connected facts. Graph traversals reveal shared skills, interests, alumni affiliation and mentoring paths directly.
- Why not copy all MySQL tables? Neo4j is a specialised projection for recommendation paths. Transactional records remain relational and authoritative.
- Why store properties on relationships? Proficiency, employment period and mentorship status describe the connection between two entities, not the shared entity itself.
- Why keep MySQL in the final loop? Capacity, availability, ratings and current business status can change; the graph can be slightly stale.
- Why an outbox? It prevents a committed MySQL update from being lost just because Neo4j or the worker is temporarily unavailable.

20.3 Glossary

Term	Meaning in this project
Node	A graph entity such as Student, Mentor, Skill or Company.
Label	A classification on a node, such as :Mentor.
Relationship	A typed, directed edge between nodes, such as :HAS_SKILL.
Property	A key-value attribute on a node or relationship.
Traversal	Following relationships from an anchor node to discover connected nodes.
Cypher	Neo4j query language used to create, match, update and return graph data.
Projection	A purpose-built representation of authoritative relational data in another store.
Transactional outbox	A record written in the same MySQL transaction as a business change and later consumed by a worker.
Idempotency	Repeating the same event or update does not create duplicate or incorrect state.
Eventual consistency	Neo4j may lag briefly, but the projection converges after successful event processing.
Explainability	Returning the relationships and evidence that support a recommendation.

Appendix A. Consolidated graph schema checklist

Use this checklist during implementation and code review.

Area	Checklist item
Identity	Stable MySQL IDs are present for every projected node.
Identity	One User node is reused when one account has both Student and Mentor profiles.
Labels	The graph uses the agreed labels consistently: User, Student, Mentor, Skill, Domain, Company, Department.
Relationships	Relationship directions and names match the approved vocabulary.
Properties	Skill proficiency, employment period and mentorship metadata are stored on relationships.
Uniqueness	Unique constraints exist for every node identity property.
Imports	MERGE is used for idempotent writes; mutable data is applied with SET.
Synchronization	Profile updates write an outbox event in the same MySQL transaction.
Synchronization	Worker retries, idempotency and source-version ordering are tested.
Recommendations	Neo4j produces candidate IDs and evidence; MySQL filters final eligibility.
Recommendations	Capacity, current availability and final allocation are never trusted to a stale graph projection.
Privacy	Passwords, tokens and unnecessary private user fields are excluded from the graph.
Performance	Queries anchor on stable IDs, use bounded patterns and are measured with PROFILE.
Recovery	The graph can be rebuilt from MySQL and outbox processing resumes after failure.

Appendix B. Useful Cypher inspection commands

Example 17. Operational graph inspection

```
// Count nodes by label
MATCH (n)
UNWIND labels(n) AS label
RETURN label, count(*) AS nodes
ORDER BY label;

// Count relationships by type
MATCH ()-[r]->()
RETURN type(r) AS relationshipType, count(*) AS relationships
ORDER BY relationshipType;

// Find students with no skill edges
MATCH (s:Student)
WHERE NOT (s)-[:HAS_SKILL]->(:Skill)
RETURN s.studentId;

// Find mentors with no domain specialisation
MATCH (m:Mentor)
WHERE NOT (m)-[:SPECIALIZES_IN]->(:Domain)
RETURN m.mentorId;

// Find mentorship history with unknown endpoint IDs
MATCH (s:Student)-[r:MENTORED_BY]->(m:Mentor)
RETURN s.studentId, r.allocationId, m.mentorId, r.status
ORDER BY r.startDate DESC;
```

Appendix C. Notes on Cypher examples

All examples are templates and must be parameterised by the FastAPI Neo4j driver. Validate date/time formats and enum values in the application before sending the query. Confirm the final query syntax against the deployed Neo4j major version. Some schema constraint types (especially existence, type and key constraints) are edition/version dependent; the examples in the main body use uniqueness constraints.

Source basis and implementation references

Primary project source: Alumni-Student Mentorship Platform: DBMS Project Blueprint, supplied in this conversation. This document follows its application architecture, database responsibility split, Neo4j section and recommendation principles.

Blueprint location	Design elements used here
Pages 4-8	Modular monolith and polyglot persistence; MySQL, Redis and Neo4j responsibilities; source-of-truth principle.
Pages 11-13	Common User identity, Student/Mentor profiles, and separation of booking from session.
Pages 14-20	Neo4j node concepts, named relationships, relationship properties, sample matching dimensions, explainability and outbox synchronization.
Pages 28-30	DBMS Unit 4 demonstration scope and the recommendation for graph traversal across skills, domains, industry and mentorship relationships.

Cypher syntax verification: Neo4j Cypher Manual pages for uniqueness constraints and MERGE semantics were consulted when preparing the implementation examples. Consult the manual for the exact Neo4j version deployed by the project before applying schema changes.

Neo4j Cypher Manual - Create constraints | Neo4j Cypher Manual - MERGE

Final implementation recommendation
Build and test the graph MVP around shared skills and domain interests first. Add alumni department and company paths next. Integrate confirmed MENTORED_BY history only after the MySQL allocation workflow is stable. The strongest project demonstration is an explainable recommendation that is discovered by Neo4j and then correctly accepted or rejected by MySQL eligibility checks.